

Website Architecture & Infrastructure

How 956woodworks.com is built, wired, deployed, and maintained — pages, image pipeline, APIs, hosting, and the runbook for future changes.

LIVE SITE

<https://956woodworks.com>

REPOSITORY

github.com/dlopez2392/956-Woodworks

HOSTING

Vercel (static, no build step)

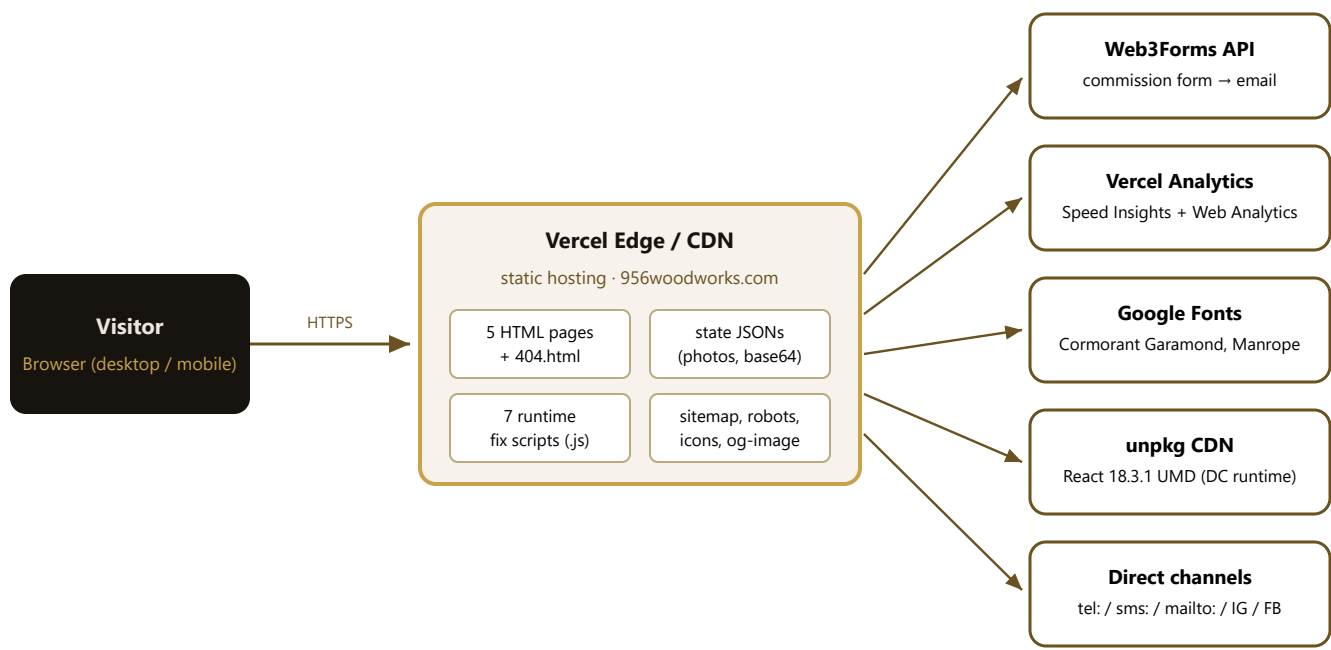
**DOCUMENT
DATE**

July 7, 2026

1. System Overview

956woodworks.com is a **fully static marketing site** for a custom woodworking business in the Rio Grande Valley. There is no backend server, no database, and no build step: five self-contained HTML pages plus a 404 page are served directly from Vercel's CDN. Interactivity (galleries, the commission form, the photo lightbox) runs entirely in the visitor's browser; the only write path out of the site is the contact form, which posts to the third-party **Web3Forms** API and arrives as email.

The pages were designed in a visual editor ("DesignCanvas" / DC) whose export format shapes the whole architecture: each exported page embeds its own copy of the DC runtime, page markup, and component logic as **gzip-compressed base64 blobs** inside the HTML, hydrated client-side with React 18. Because that bundle cannot be edited directly, all post-export fixes and features are delivered as small, network-loaded **runtime scripts** (section 7).



browser-side calls only — the site itself has no server code

Figure 1 — High-level architecture. Everything is static files on Vercel; all API traffic originates in the visitor's browser.

2. Technology Stack & Page Anatomy

Layer	Technology	Notes
Authoring	DesignCanvas (DC) visual editor	Source files: 956 Woodworks.dc.html , Gallery.dc.html , 3 care-page sources at repo root
Page runtime	React 18.3.1 (UMD via unpkg) + DC engine	DC engine + page markup shipped inside each HTML as gzipped-base64 <code>__bundler</code> manifest blobs
Components	Custom elements (<code><image-slot></code>)	Shadow-DOM photo frames with cover-fit + reframe math; 87 slots across home and gallery
Styling	Inline styles + embedded CSS	Dark walnut/gold brand (#17130F / #C9A24B); Cormorant Garamond (serif) + Manrope (sans)
Enhancements	7 vanilla-JS runtime scripts	Loaded with <code>defer</code> before <code></body></code> ; see section 7
Hosting	Vercel static deployment	Project root = <code>site/</code> ; no framework, no build command
Version control	GitHub, single <code>main</code> branch	Push to main = production deploy (Vercel Git integration)

Anatomy of an exported page

Each `site/*.html` file contains, in order: a hand-maintained static `<head>` (SEO meta, JSON-LD, preload hints, an inline fetch-redirect script, a head-restore script); the DC bundle (React loader + `<script`

type="__bundler/manifest"> blobs holding the compressed runtime, markup, and component logic); and the runtime fix scripts plus analytics tags before </body> . At load time the DC engine decompresses its blobs, mounts the page with React, and **replaces the entire <head> and <html> attributes** — which is why an inline capture-and-restore script re-applies the SEO tags and a11y.js re-asserts lang="en" .

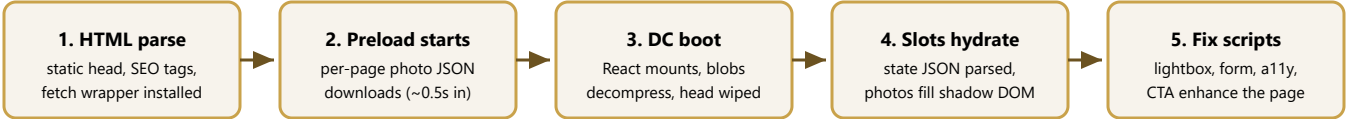


Figure 2 — Page boot sequence.

3. Hosting, Domain & Deployment Pipeline

Item	Value
Vercel project	danlopez508-8452s-projects/956woodworks — deploys the <code>site/</code> folder as-is (static, ~5s deploys)
Production domain	956woodworks.com (apex). <code>www</code> 308-redirects to apex via <code>vercel.json</code> host rules
Domain registrar / DNS	Registered at GoDaddy; nameservers point to Vercel DNS (records managed via <code>vercel dns</code> , not GoDaddy)
Deploy trigger	Git push to <code>main</code> → Vercel Git integration auto-deploys to production
Access	Deployment Protection disabled (public site, no Vercel SSO gate)
Security headers	<code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: SAMEORIGIN</code> , <code>Referrer-Policy: strict-origin-when-cross-origin</code> , restrictive <code>Permissions-Policy</code> (all via <code>vercel.json</code>)
Legacy redirects	Old <code>*.dc.html</code> URLs 308-redirect to their kebab-case pages (safety net for re-exports and old bookmarks)
Custom 404	<code>site/404.html</code> , branded, served automatically by Vercel for unknown paths

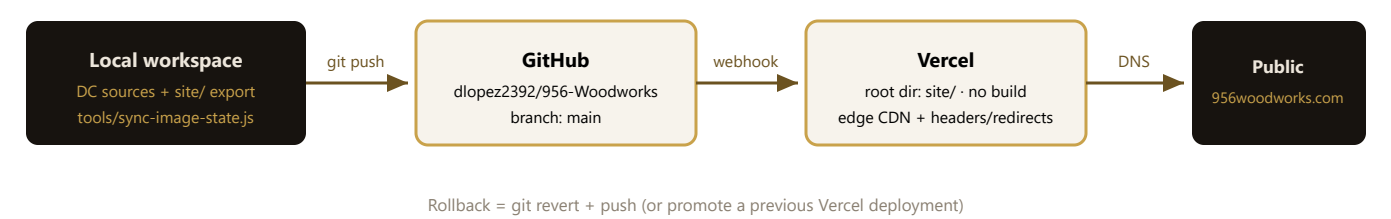


Figure 3 — Deployment pipeline. A push to main is live in under a minute.

4. The Image-Slot Photo System (core subsystem)

All 87 photos live as **base64 data-URLs inside JSON "sidcar" files**, not as image files. Each `<image-slot id="...">` element fetches the sidcar, looks up its id, and renders the photo in shadow DOM with cover-fit and reframe values (`{u, s, x, y}` = image, scale, x/y offsets in frame-%). The bundled loader **rejects any non-data:image/ URL**, so photos must stay base64. Slot ids are permanent — reusing or renumbering one silently swaps photos (this caused real incidents; rule is codified in `CLAUDE.md`).

For performance, the editor's single master file is split per page and recompressed by `tools/sync-image-state.js` . An inline `<head>` script wraps `window.fetch` to redirect the runtime's request for the full file to the

page-specific one (falling back to the full file if missing) and caches the response, because the bundle evaluates the loader module twice on the gallery page.

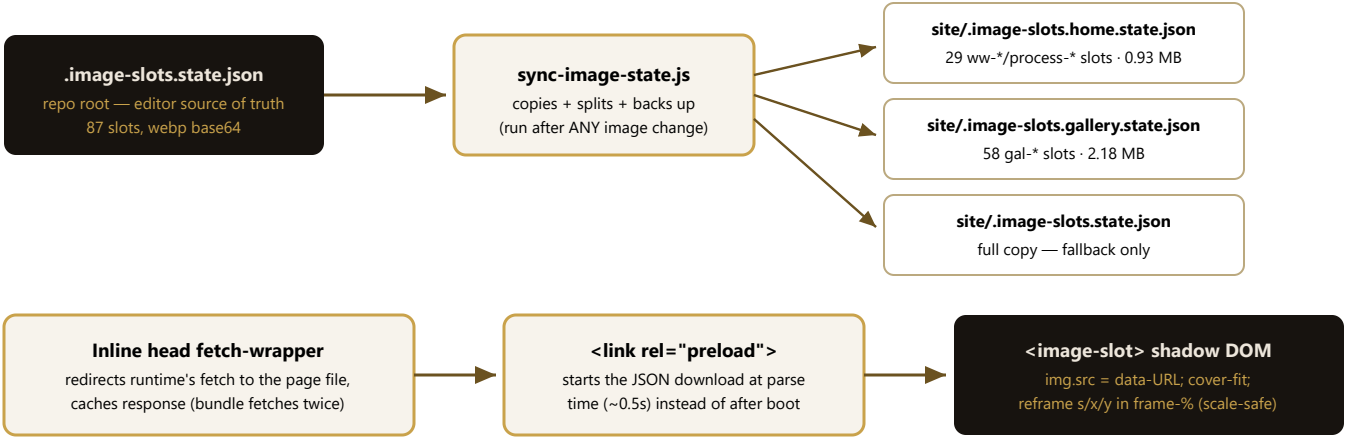


Figure 4 — Photo pipeline: authoring file → sync tool → per-page sidecars → browser hydration.

Performance history. The original export shipped one 5.7 MB sidecar to every page (the gallery downloaded it twice = 11.4 MB). After converting 21 JPEG/PNG photos to WebP q80, downscaling 23 oversized photos to 660 px, splitting per page, deduplicating the double-fetch, and preloading: the home page now transfers **~640 KB** and the gallery **~1.7 MB** (Brotli-compressed on the wire). Lighthouse (mobile): Accessibility **100**, SEO **100**, Best Practices 96, Performance 42 — the last is bound by the DC client-side rendering architecture, not asset weight.

5. APIs & External Services

Service	Endpoint / integration	Direction	Purpose & wiring
Web3Forms	POST <code>https://api.web3forms.com/submit</code> (JSON)	Browser → out	Commission form. <code>form.js</code> intercepts the Send click in capture phase (blocking the old <code>mailto:</code>), validates, posts <code>access_key</code> , <code>name</code> , <code>email</code> , <code>piece_type</code> , <code>budget</code> , <code>message</code> ; Web3Forms emails <code>956woodwork@gmail.com</code> . Key (public by design, domain-scoped): <code>d4fe...5338</code> . No CAPTCHA yet; add hCaptcha via Web3Forms if spam appears.
Vercel Speed Insights	<code>/_vercel/speed-insights/script.js</code> + POST <code>/_vercel/speed-insights/vitals</code>	Browser → out	Core Web Vitals field data. Script on all 5 pages; verified live (vitals POST returns 200). Dashboard: project → Speed Insights tab.
Vercel Web Analytics	<code>/_vercel/insights/script.js</code>	Browser → out	Cookieless page-view analytics. Script tags in place on all pages, but the project-level toggle is not yet active (route 404s). Enable: project → Analytics tab, then redeploy.
Google Fonts	<code>fonts.googleapis.com</code> / <code>fonts.gstatic.com</code>	Browser ← in	Cormorant Garamond + Manrope, <code>display=swap</code> , preconnected. (Home page also carries fonts inlined by the DC export.)
unpkg	<code>unpkg.com/react@18.3.1</code> , <code>react-dom@18.3.1</code>	Browser ← in	React UMD builds required by the DC runtime bundle.
Google Search Console	Domain property <code>sc-domain:956woodworks.com</code>	Config	Verified via TXT record in Vercel DNS; sitemap submitted at the apex.
Direct contact	<code>tel:</code> / <code>sms:9562921696</code> / <code>mailto:</code> / <code>instagram.com/956woodworks</code> / Facebook page	Links	Call pill (header), Call + Text Me + DM buttons (contact section), footer links.

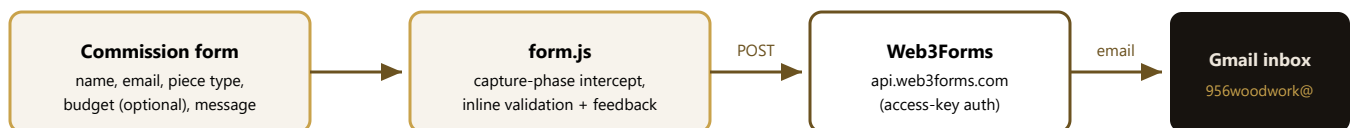


Figure 5 — Lead flow. The only data leaving the site is a form submission, delivered as email.

6. Pages & SEO

Page	URL	Structured data & notes
Home	/	LocalBusiness JSON-LD (services catalog, RGV area served) + FAQPage; hero, before/after slider, categories, process, 18-photo grid, testimonials (placeholders), FAQ accordion, commission form
Gallery	/gallery.html	BreadcrumbList; 58 photos, category filter chips, lightbox with captions
Care guides (3)	/cutting-board-care.html , /furniture-care.html , /rolling-pin-care.html	Article + BreadcrumbList JSON-LD; editorial layout ending in a commission CTA
Not found	any unknown path	404.html , noindex, branded, links back to home/gallery/contact

Site-wide: per-page titles/descriptions/canonicals, Open Graph + Twitter cards with a 1200×630 share image, SVG favicon + 180×180 `apple-touch-icon.png` , `sitemap.xml` (5 URLs) and `robots.txt` , `lang="en"` . Alt text for all photos is applied at runtime by `a11y.js` (gallery captions / curated map).

7. Runtime Fix Scripts (the "patch layer")

Because the DC export embeds its logic in compressed blobs, the live site is enhanced by small network-loaded scripts. Convention: every fix also patches the DC *source* files where possible so a future re-export bakes it in; injected DOM carries ids and is re-applied by slow maintenance ticks; event handlers are delegated to `document` (the DC engine re-renders every few seconds and drops per-node listeners, but keeps injected nodes).

Script	Pages	What it does
<code>compare-slider.js</code>	home	Wires the before/after slider drag (export didn't bind the pointer handler)
<code>mobile-scroll-fix.js</code>	home, gallery	Overrides <code>touch-action:none</code> on photos so mobile scrolling works
<code>lightbox.js</code>	home, gallery	Tap-to-enlarge photo viewer: prev/next, keyboard, counter, category captions, alt text
<code>form.js</code>	home	Web3Forms submission + validation; injects budget dropdown, photo-texting hint, Text Me button
<code>faq.js</code>	home	Injects the 9-question FAQ accordion section (delegated toggle handlers)
<code>a11y.js</code>	home, gallery	Photo alt text, <code>lang="en"</code> re-assert, form aria-labels, footer contrast fix, removes dead credit anchors, hides decorative slots
<code>ui-polish.js</code>	home, gallery	Sticky mobile "Commission a piece" pill; mobile header spacing fix

8. Repository Layout

Path	Contents
/ (root)	DC editor sources (*.dc.html), master .image-slots.state.json + .backup , image-slot.js component source, uploads/ (original photos, not used at runtime), CLAUDE.md (persistent project rules)
/site	The deployed website: 5 pages + 404, per-page state JSONs, 7 runtime scripts, vercel.json , sitemap/robots/icons/og-image, .vercel link (gitignored)
/tools	sync-image-state.js — the photo pipeline (copy → optional recompress → split → backup)
/docs	This document (HTML source + PDF)

9. Maintenance Runbook

Golden rule №1 — slot ids are permanent. Never renumber, reorder, or reuse an `<image-slot>` id. New photos get brand-new ids (bump a counter: `gal-226` , `gal-227` ...). Ids are the only key into the photo store.

Golden rule №2 — after ANY image change, run `node tools/sync-image-state.js` . The live pages read the per-page files; the full-file fallback only covers a *missing* page file, not a *stale* one. Skipping this shows old photos on the live site.

Common tasks

- **Deploy anything:** commit and push to `main` . Live in ~30–60s. Verify at 956woodworks.com.
- **Add/replace a photo:** edit in the DC editor (writes the root state file) → run the sync tool → push.
- **Change copy/layout:** edit the DC source, re-export, then work the re-export checklist below.
- **Rollback:** `git revert` the commit and push (or promote an older deployment in the Vercel dashboard).

Re-export checklist (whenever `site/` is regenerated from the DC editor)

- Copy `image-slot.js` into `site/` (the export drops it).
- Fix the 5 nav links in `site/index.html` (`.dc.html` names → kebab-case pages) — or rely on the `vercel.json` redirects.
- Re-add the `defer` script tags for the 7 runtime scripts + the two Vercel analytics tags before `</body>` .
- Confirm the inline head scripts (SEO restore, fetch-wrapper, preload link) survived — they live in the source helmets, but verify.
- Run `node tools/sync-image-state.js` ; keep `404.html` , `vercel.json` , `sitemap/robots/icons` in place.

Known limitations & watch items

- Mobile Lighthouse performance (~42) is bound by DC's client-side full-page rendering (~6s main-thread on a throttled phone). Fixing it means re-platforming; real-world fast connections are fine (CLS = 0).
- Web Analytics toggle pending in the Vercel dashboard (script route 404s until enabled + redeployed); this is also the only remaining Lighthouse Best-Practices deduction (console 404).
- Testimonials are anonymous placeholders — swap in real quotes (name + city) when available; never fabricate.
- Third-party dependencies at runtime: unpkg (React) and Google Fonts. If unpkg is ever slow/down, pages degrade badly — consider self-hosting React if it becomes an issue.
- If form spam appears: enable hCaptcha in Web3Forms (key is domain-scoped).

Prepared July 7, 2026 · Reflects production state at commit `49aa532` · Sources: repository `dlopez2392/956-Woodworks`, Vercel project `956woodworks`, live-site verification.